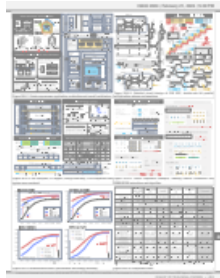




1



2



3

ISSCC 2024 / SESSION 30 / DOMAIN-SPECIFIC CO

30.6 Vecim: A 289.13GOPS/W RISC-V Vector Co-Processor Compute-in-Memory Vector Register File for Efficient High-Performance Computing

Yipeng Wang, Mengtian Yang, Chieh-pu Lo, Jaydeep P. Kulkarni

University of Texas, Austin, TX

Vector processors have re-emerged in high-performance computing and flagship SoC designs for their improved programmability, appealing power efficiency, and high throughput compared to general-purpose CPUs and GPUs. For modern data-parallel (ML, DSP, vision), vector processors can approach the efficiency of custom ASICs while maintaining the flexibility of CPUs and GPUs. However, there are still limitations to the widespread adoption of vector architectures in general-purpose systems due to expensive on-chip data movement, the high off-chip memory bandwidth requirement, and the Vector Register File (VRF) complexity.

This work, Vecim, presents Compute-in-Memory (CIM) as a microarchitecture to address the above limitations. A 1R1W SRAM VRF supporting all-digital INT8/BF16/FP16 multiplication and addition is first presented, eliminating movement between the VRF and ALU/FPU without increasing the complexity of the VRF organization. The array processing feature added to the vector processor also opens new architectural opportunities for parallelism, data reuse and performance. A CIM-aware vector sequencer is proposed for instruction and data-level parallelism. Vector instruction extensions for further data reuse.

Figure 30.6.1 shows the overall architecture based on the baseline design. The vector coprocessor receives instructions from a scalar CPU. The vector sequencer fetches the instruction and sends it to the corresponding execution unit. The vector store unit (LSU) decodes, generates memory requests and loads/stores vector data to a DRAM interconnect, which has a throughput of 32b per lane per cycle. The LSU is responsible for special slide instructions that access all lanes simultaneously. For main arithmetic operations happen in vector lanes, which consist of a VRF request handler, 8 VRF banks with 4Kb capacity each, an interconnect, and a 64b results/cycle throughput. The CIM VRF executes all INT8/BF16/FP16 related instructions in memory, and other arithmetic tasks are routed to the datapath for processing by the ALU/FPU. The RISC-V Vector (RVV) ISA features 16 registers. With four lanes implemented, this design features a 4Kb physical register length capable of 64 DP or 512 INT8 data in a physical register.

8T 1R1W SRAM is generally used for register files due to its decoupled read/write for higher throughput and lower Vmin to be compatible with core logic. Traditional CIM with adder trees requires extensive modifications to the SRAM array, incurring a timing penalty and large area overhead. On the other hand, bitline (BL) comparison-based bit-serial multiplication has low throughput and energy efficiency. Additionally, register files can only provide a bit-aligned data layout for all operands, invalidating the dataflow [3]. Figure 30.6.2 illustrates the proposed double-rate-bit-parallel multiplication (DBPM), to achieve high throughput in-memory multiplication without modifying the data flow in the VRF. In this scheme, both operands are retained in register file in their original bit organization. First, these bit operands undergo bitwise multiplication. The resulting product is routed to dedicated registers near the memory. One of the registers is then shifted by one, and two 13b adders accumulate the bit-wise result. Due to the staggered bit layout of the two consecutive multiplications, they are grouped in a way that doubles the throughput. For INT8, it takes 4 cycles to complete a